

Firewall Normal Form: A Normalization Framework for Firewall Management and Anomaly Correction

Chang-Sheng Lee¹ and Ling-Jyh Chen^{1,2}

¹*Institute of Information Science, Academia Sinica*

²*Research Center for Information Technology Innovation, Academia Sinica*

{johnsonlee911205, ccljj}@iis.sinica.edu.tw

Abstract—Firewalls are widely deployed as the first line of defense in cybersecurity. However, as firewall rule sets evolve over time, overlaps in the matching space inevitably arise, leading to unintended behavior. Existing approaches to anomaly detection and correction often require manual intervention. In addition, the high complexity of rule interactions often discourages administrators from modifying policies for fear of inadvertently disrupting existing services. In this paper, we propose a series of Firewall Normal Forms (FNF) to characterize the structural completeness and organization of a firewall. By progressively enforcing these normal forms, order dependency is eliminated, and anomalies are removed. To validate the proposed approach, we conducted experiments on a real-world firewall dataset under both static and dynamic conditions. We measured how different levels of FNF affect rule set characteristics, including the number of rules, the distribution of anomalies, and the anomaly detection time. The results show that higher normal forms exhibit advantages in anomaly detection time and overall management efficiency.

Index Terms—Firewall, Firewall Rule Anomaly, Anomaly Correction, Normalization

I. INTRODUCTION

Firewalls are widely deployed to defend against cyberattacks. However, firewall rule sets tend to grow over time. In many cases, the original intent is lost or can no longer be traced. As a result, overlaps in the matching space among rules become difficult to avoid. When misconfigurations cause rule semantics to be altered by order dependency, the original security objectives cannot be achieved.

To systematically address firewall misconfigurations, Al-Shaer and Hamed proposed a formal model to classify policy anomalies [1], defined as situations where a single packet matches multiple firewall rules, resulting in ambiguous filtering behavior. Based on the definition, tools such as Firewall Policy Advisor and Fireman have proposed anomaly detection methods to solve this problem [1], [2]. Anomaly correction tools, such as those proposed by Abedin et al. and Hu et al. [3], [4], have also been developed to assist in resolving anomalies. However, these methods have seen limited adoption in practice. One primary reason is the lack of a unified semantic foundation, as vendors adopt different syntax and matching

semantics [5], [6], [7]. This challenges the establishment of a consistent analytical framework across environments. In addition, most existing approaches require manual intervention. However, complex interactions among rules often discourage administrators from performing cleanup, as they fear that modifying rules may inadvertently disrupt existing policies. If rule order dependency cannot be eliminated and rules cannot be atomized, a Zero Trust architecture cannot be realized.

In this paper, we propose a normalization-based approach to address the limitations of previous methods. Inspired by database normalization [8], [9], we introduce the Firewall Normal Form (FNF), which defines a series of normal forms to characterize firewall rule sets at different levels of normalization, each corresponding to a set of constraints that must be satisfied. By progressively enforcing these constraints, semantic overlaps among rules are eliminated and rule sets are transformed into a representation that is structurally explicit and semantically independent. We further implement the four normalizations and evaluate their practical applicability by applying FNF to a real-world dataset, demonstrating its advantages under dynamic policy evolution.

We summarize our main contributions as follows:

- We define a series of Firewall Normal Forms that eliminate order dependency and anomalies as higher normal forms are reached, thereby addressing the gap in prior approaches that rely on manual intervention.
- We design a vendor-neutral intermediate representation for firewall rules and implement parsers for Fortinet FortiGate, enabling cross-environment policy analysis and facilitating broader practical adoption.
- We are among the few studies that empirically analyze firewall policies using real-world, non-synthetic datasets, providing stronger evidence beyond prior work based on synthetic data. We characterize anomaly distributions and quantify the impact of FNF on detection efficiency and firewall management under evolving rule sets.

The remainder of this paper is organized as follows. Section II reviews anomaly classifications, existing resolution approaches, and the database normalization referenced by FNF. Section III defines the four Firewall Normal Forms and normalization procedures. Section IV evaluates the proposed approach. Section V concludes the paper.

This research was supported by the National Science and Technology Council of Taiwan under Grant: NSTC 112-2221-E-001-005-MY3. The authors would also like to thank the Department of Information Technology Services, Academia Sinica, for providing the firewall dataset.

II. RELATED WORK

In this section, we review the definition of anomalies and different methods to analyze them, highlighting the limitations of existing methods. We also review database normalization as the foundation of the Firewall Normal Form.

A. Firewall Rule Anomalies

A firewall rule anomaly refers to unintended behavior arising from overlapping rule conditions. In a typical firewall, an Access Control List is enforced as a packet filter, where each rule consists of conditions and an associated action [10], [11], [12]. In such a system, packets are evaluated sequentially against rules, and the first matching rule determines the action. However, when rule conditions overlap, a packet may match multiple rules, making the final action dependent on their relative order. Such order-dependent behavior may result in anomalies that undermine policy correctness and prevent certain rules from achieving the intended enforcement objectives.

Al-Shaer and Hamed employed a formal method to derive all possible relationships that may exist between any pair of firewall rules [1]. Based on this theoretical foundation, they defined firewall rule anomalies and classified them into the following four types.

- 1) **Shadowing:** A rule is shadowed when an earlier rule with a different action matches all packets that would otherwise match the rule, rendering the rule ineffective.
- 2) **Correlation:** Two rules with different actions are correlated if they match some of the same packets, but neither fully covers the other.
- 3) **Generalization:** A rule generalizes a preceding specific rule if it has a different action and covers all its packets.
- 4) **Redundancy:** A redundant rule performs the same action on the same packets as another rule, such that removing it has no effect on the policy.

Anomalies are not always errors. They can be intentionally introduced. For example, a generalization anomaly often appears when a broad rule is modified by adding an exception.

B. Anomaly Detection and Anomaly Correction

Firewall anomaly detection and correction have been extensively studied to help resolve configuration inconsistencies. To help identify inconsistencies, various anomaly detection methods have been proposed. A representative work is Firewall Policy Advisor [1], which compares all fields between a pair of rules and determines the anomaly type based on predefined logical rules. Subsequent work has extended these methods to support more types of firewalls by incorporating analysis for stateful firewalls and expanding anomaly detection across multiple firewalls [13], [14]. In contrast to approaches that focus on extending functionality, a survey categorizes methods that improve efficiency as model-based approaches [5]. FIREMAN is one such example [2], accelerating anomaly detection by conducting comparisons between each rule and all preceding rules collectively. Recent studies have also indicated that machine learning-based anomaly detection holds significant potential [15]. Moreover, we showed that LLM-based

approaches can achieve detection performance comparable to algorithmic methods while also providing interpretable explanations and potential corrections [16].

Anomaly correction goes a step further by aiming to resolve these anomalies. Early work focused on suggesting fixes rather than automatically resolving anomalies. For example, Firewall Policy Editor flags anomalies triggered by rule modifications, leaving decisions to administrators [1]. Subsequent research introduced automated anomaly correction to reduce manual effort. Abedin addressed anomalies by reconstructing the rule set through incremental rule insertion, where each newly inserted rule is compared with existing rules and anomalies are resolved on the fly [3]. When the new rule and an existing rule are equal, the rule with the reject action is retained if present; otherwise, either rule may be retained as they are equivalent. If the new rule is a subset of an existing rule, it is inserted before that rule. If the two rules are correlated, they are resolved by splitting the rules. Hu et al. proposed a grid-based matrix representation to identify interdependent regions within the packet space [4]. These regions are partitioned into Conflict Correlation Groups that group mutually dependent conflicts. The approach further derives Action Constraints to explicitly specify the expected behavior within each conflicting region. Finally, a greedy algorithm reorders the rules to satisfy these constraints as much as possible while minimizing conflicts.

Nevertheless, these methods cannot automatically eliminate all anomalies. They typically require manual intervention or do not guarantee complete anomaly removal via rule reordering.

C. Database Normalization

Database normalization was introduced by Codd as part of the relational model to improve structural organization and reduce redundancy and inconsistencies [9]. In the original formulation, normalization denoted atomizing multivalued data, allowing relations to be represented as two-dimensional arrays without requiring complex data structures. In subsequent work, Codd formally defined a hierarchy of normal forms, renaming the original concept First Normal Form (1NF) [17]. He further introduced higher normal forms to eliminate different types of dependencies. Second Normal Form (2NF) removes partial dependencies, thereby eliminating redundancy caused by attributes that depend only on part of a composite key. Third Normal Form (3NF) removes transitive dependencies, so that each attribute depends only on the key rather than on other non-key attributes. Boyce–Codd Normal Form (BCNF) further eliminates redundancy by requiring that every determinant be a superkey [18]. Fagin extended Codd’s work by introducing Fourth Normal Form (4NF) to eliminate multivalued dependencies and Fifth Normal Form (5NF) to decompose relations into minimal forms under lossless decomposition [19], [20].

A database query is similar to the firewall packet matching. In the former, multiple representations of the same data can lead to inconsistent query results or anomalies. In the latter, overlaps in rules may cause a single packet to correspond to multiple conflicting decisions. To ensure consistency in matching, firewall rule sets also require a normalization framework.

III. FIREWALL NORMAL FORM

To establish a foundation for transforming firewall rule sets into an order-independent representation, we introduce the Firewall Normal Forms (FNF) inspired by database normalization. These normal forms characterize the degree of structural decomposition, semantic clarity, and elimination of inter-rule dependencies. Based on the definitions of FNF, a sequence of normalization transformations can be applied to achieve higher normal forms, progressively reducing ambiguity, redundancy, and hidden dependencies.

A. First Firewall Normal Form (1FNF)

Definition. A set of firewall rules satisfies the First Firewall Normal Form (1FNF) if and only if each field satisfies the atomicity condition. Here, atomicity means that each field is expressed solely in terms of primitive values defined over its domain, such as basic intervals or discrete atomic elements, without the use of aliases or non-standard representations.

Effect. The goal of 1FNF is to eliminate syntactic heterogeneity, enabling rules to be analyzed under a unified syntax and representation. By enforcing atomicity, each rule represents a single, well-defined matching condition. This objective is consistent with the notion of atomicity in First Normal Form in database normalization.

Algorithm 1 presents the process of transforming an unnormalized dataset into one that satisfies 1FNF. First, action and matching conditions are extracted from the original rule syntax and mapped to a unified JSON representation, enabling comparison and set operations across heterogeneous rule formats. For matching conditions, only five fields are retained for packet matching: *source interface*, *destination interface*, *source address*, *destination address*, and *service* defined by *port* and *protocol*. The representation is then further standardized by resolving group and alias into their underlying members. Different address representations, such as CIDR, ranges, and subnets, are normalized to a single range-based form. The atomization process is performed after syntax unification. A Cartesian product expansion is applied over all fields, decomposing the rule into multiple rules, each containing a single combination of atomic conditions.

B. Second Firewall Normal Form (2FNF)

Definition. A set of firewall rules satisfies the Second Firewall Normal Form (2FNF) if and only if it satisfies 1FNF and no rule is rendered ineffective due to complete coverage by another rule.

Effect. The goal of 2FNF is to eliminate redundant and shadowed rules that do not contribute to effective filtering. This objective is similar to that of Second Normal Form in database normalization, which reduces redundancy by eliminating partial dependencies that do not contribute to the unique identification of a rule.

Whether a rule makes no effective contribution to the overall policy can be determined by performing pairwise comparisons across the rule set. For each pair of rules, their containment

Algorithm 1 First Firewall Normalization

Input: Raw firewall configuration D

Output: Rule list R_1

```

1: parse  $D$  to get  $addr\_map, svc\_map, zone\_map$ 
2:  $R_1 \leftarrow \emptyset$ 
3: for all  $r \in D$  do
4:   extract  $F = \{srcintf, dstintf, srcaddr, dstaddr, service\}$ 
5:   extract  $action$ 
6:    $F' \leftarrow \emptyset$ 
7:   /* convert syntax into a unified representation */
8:   for all  $f \in F$  do
9:      $E \leftarrow \emptyset$ 
10:    for all token  $x \in f$  do
11:      for all  $M \in maps(f)$  do
12:        if  $x \in M$  then
13:           $E \leftarrow E \cup M[x]$ 
14:          break
15:        if  $x \notin \bigcup maps(f)$  then
16:           $E \leftarrow E \cup \{x\}$ 
17:       $F'[f] \leftarrow E$ 
18:      Standardize the representation of  $F'[f]$ 
19:    /* atomize value */
20:     $C \leftarrow \times_{f \in F} F'[f]$ 
21:    for all  $(i, j, a, b, s) \in C$  do
22:       $R_1 \leftarrow R_1 \cup \{(i, j, a, b, s, action)\}$ 
23: return  $R_1$ 

```

Algorithm 2 Second Firewall Normalization

Input: Rule list R_1

Output: Rule list R_2

```

1:  $S \leftarrow \emptyset$ 
2: for all  $r_i, r_j \in R_1$  with  $i < j$  do
3:   if  $r_j \subseteq r_i$  then
4:      $S \leftarrow S \cup r_j$ 
5:  $R_2 \leftarrow R_1 \setminus S$ 
6: return  $R_2$ 

```

relationship is examined. If one rule is a subset of a higher-priority rule, meaning that the values of all its fields are fully contained within the corresponding fields of the other rule, it is identified as redundant and recorded. To avoid unintended effects caused by dynamic changes in rule ordering during the evaluation process, all identified redundant rules are removed in a single batch after the comparison phase. The overall procedure is summarized in Algorithm 2.

C. Third Firewall Normal Form (3FNF)

Definition. A set of firewall rules satisfies the Third Firewall Normal Form (3FNF) if and only if it satisfies 2FNF and the matching spaces of any two distinct rules are mutually exclusive.

Algorithm 3 Third Firewall Normalization

Input: Rule list R_2 **Output:** Rule list R_3

```

1:  $R_3 \leftarrow R_2$ 
2:  $updated \leftarrow \text{true}$ 
3: while  $updated = \text{true}$  do
4:    $updated \leftarrow \text{false}$ 
5:   for all  $r_i, r_j \in R_3$  with  $i < j$  do
6:     if  $r_i \cap r_j \neq \emptyset$  then
7:        $r_{ov} \leftarrow r_i \cap r_j$ 
8:        $r_{ov}[action] \leftarrow r_i[action]$ 
9:        $r_{i\_new} \leftarrow r_i \setminus r_{ov}$ 
10:       $r_{j\_new} \leftarrow r_j \setminus r_{ov}$ 
11:       $R_3 \leftarrow (R_3 \setminus \{r_i, r_j\}) \cup r_{i\_new} \cup r_{ov} \cup r_{j\_new}$ 
12:       $updated \leftarrow \text{true}$ 
13:    break
14: end while
15: return  $R_3$ 

```

Effect. The goal of 3FNF is to eliminate the order dependency in a firewall. By decomposing overlapping packet-matching spaces among rules, the behavior of each rule is independent of others. As a result, each packet is associated with exactly one rule that uniquely determines its action, and its behavior is independent of all other rules. This objective parallels the removal of transitive dependencies in database normalization, where all attributes are ensured to depend only on the key rather than on other non-key attributes.

To decompose overlapping rule packet-matching spaces, the intersection between two rules is explicitly computed and split into three disjoint parts: the overlap and the two residual non-overlapping regions. Algorithm 3 provides the detailed implementation. The algorithm starts by iterating all rule pairs and checking whether their matching spaces intersect. If an overlap is detected, the overlapping packet-matching space is extracted as a separate rule, which is assigned the action of the higher-priority rule to preserve the original firewall semantics. At the same time, the overlapping space is removed from the matching spaces of both rules. Since newly generated rules may still overlap with other rules, this process is repeated iteratively until all rules become mutually disjoint.

D. Fourth Firewall Normal Form (4FNF)

Definition. A set of firewall rules satisfies the Fourth Firewall Normal Form (4FNF) if and only if it satisfies 3FNF and is minimal in size while preserving the same policy semantics.

Effect. The goal of 4FNF is to merge the fragments of the decomposed rules into a coherent form, ensuring that the semantics of the rule set remain unchanged while each rule is represented in its simplest atomic form. This objective parallels Fifth Normal Form in database normalization, which achieves a minimal form under lossless decomposition.

Given a rule set that has already been transformed into 3FNF through rule decomposition, Algorithm 4 is further applied

Algorithm 4 Fourth Firewall Normalization

Input: Rule list R_3 **Output:** Rule list R_4

```

1:  $R_4 \leftarrow R_3$ 
2:  $updated \leftarrow \text{true}$ 
3: extract  $F = \{srcintf, dstintf, srcaddr, dstaddr, service\}$ 
4: while  $updated = \text{true}$  do
5:    $updated \leftarrow \text{false}$ 
6:   for all  $r_i, r_j \in R_4$  with  $i < j$  do
7:     for all  $d \in F$  do
8:       if  $r_i[action] = r_j[action]$  then
9:         if  $r_i[F \setminus \{d\}] = r_j[F \setminus \{d\}]$  then
10:          if  $r_i[d] \cup r_j[d]$  is contiguous then
11:             $r_m \leftarrow r_i$ 
12:             $r_m[d] \leftarrow r_i[d] \cup r_j[d]$ 
13:             $R_4 \leftarrow (R_4 \setminus \{r_i, r_j\}) \cup \{r_m\}$ 
14:             $updated \leftarrow \text{true}$ 
15:          break
16:        if  $updated = \text{true}$  then
17:          break
18:   end while
19: return  $R_4$ 

```

to achieve a minimal, order-independent firewall. Specifically, two rules are eligible for merging if they share the same action, have identical values across all fields except for one, and the differing field can be combined while still satisfying the atomicity condition. When these conditions are met, the two rules are merged into a single rule. This process is applied iteratively until no more pairs of rules can be merged.

IV. EVALUATION

To evaluate the effectiveness of the Firewall Normal Forms (FNF), we applied FNF to a large-scale dataset deployed in a real-world environment. In addition to examining the effectiveness of applying FNF to rule sets, we also analyzed how the resulting structural properties shaped the behavior of a rule set as policies evolve. All experiments were implemented in Python and conducted on a machine equipped with an AMD Ryzen 9 6900HS CPU and 16 GB of RAM.

A. Dataset Analysis

All experiments were conducted on the firewall rule sets deployed in the Academia Sinica data center. The rule sets were written in Fortinet FortiGate CLI syntax. This firewall consists of 693 rules and follows whitelisting, in which the default action is to reject all traffic, and explicit accept rules are defined to permit specific packets to pass through the firewall. To avoid a potential data leak, the firewall rules were anonymized. Specifically, the first three octets of each IP address are encrypted, while the last octet is preserved. Since the subnet masks in the dataset are greater than or equal to /24 or multiples of 8, this anonymization does not affect the actual filtering behavior of the firewall.

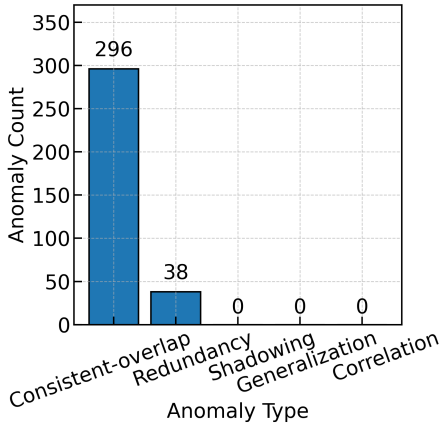


Fig. 1. Unnormalized dataset anomaly distribution

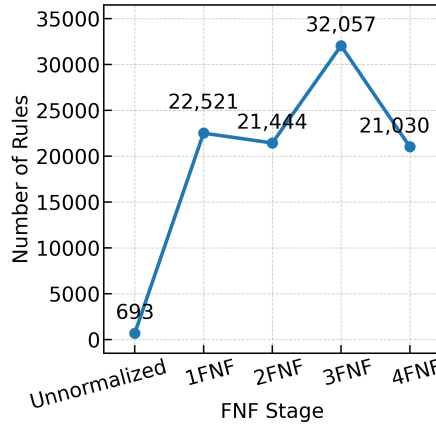


Fig. 2. Number of rules at different FNF

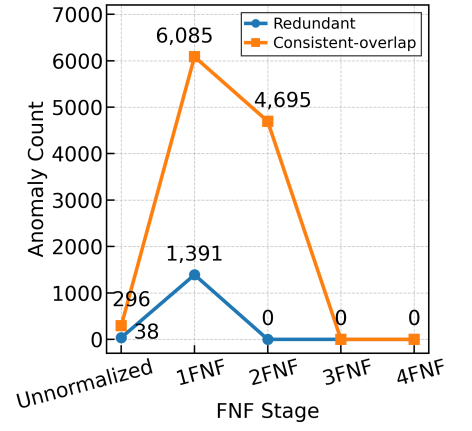


Fig. 3. Number of anomalies at different FNF

B. FNF Transformation

To examine how FNF transforms the rule set, we analyzed changes in the anomaly distribution throughout the normalization process. Before applying FNF, we first visualized the anomaly distribution in the original dataset. Although Al-Shaer defined four types of firewall rule anomalies to capture several important relationships between overlapping rules, these definitions do not cover all possible cases in which two rules share a non-empty matching space. In particular, when the actions of two overlapping rules are identical, some overlap relationships are not explicitly classified, including cases where a lower-priority rule fully covers the matching space of a higher-priority rule and cases where two rules partially overlap without either rule fully containing the other's matching. These cases are still important to our analysis, since our goal is to ensure complete mutual exclusivity among rules. As a result, we define a consistent-overlap anomaly as a relationship in which two rules share a non-empty matching space while having the same action. This additional category enables our analysis to account for all overlaps that must be removed to achieve complete mutual exclusivity among rules.

Fig. 1 presents the distribution of anomalies in the unnormalized real-world dataset. Due to the nature of whitelist firewalls, anomalies that depend on conflicting actions, such as shadowing, generalization, and correlation anomalies, were not observed in the original dataset. Instead, the observed anomalies were concentrated in same-action overlap relationships, including 296 consistent-overlap anomalies and 38 redundancy anomalies. Although the absence of conflicting-action anomalies suggested that no rule was prevented from achieving its original intent by another rule with a different action, the large number of consistent-overlap anomalies and redundancy anomalies showed that rule overlap remained prevalent. This made it difficult to determine whether each rule contributed a distinct matching space. As a result, policy verification and maintenance became complicated.

Based on the anomaly distribution in the unnormalized dataset, we further observed how the number of rules evolved

throughout the application of progressively higher normal forms and how anomalies evolved as the structural properties of the rule set changed. Fig. 2 shows the number of rules in each normal form, while Fig. 3 presents the number of anomalies observed under different normal forms. After the unnormalized dataset was transformed into one that satisfied 1FNF, the number of rules increased substantially due to atomization, where the fields were decomposed into uniform and atomic representations. As a result, the number of anomalies also increased proportionally with the expanded rule set. When the rule set was further transformed to 2FNF, all redundant rules were removed. Since some of the redundant rules overlapped with other rules, their removal also led to a reduction in both the rule count and the number of consistent-overlap anomalies. In 3FNF, the remaining consistent-overlap anomalies were eliminated by decomposing overlapping rules into mutually exclusive fragments, resulting in an anomaly-free rule set. Although this decomposition increased the number of rules, 4FNF subsequently merged compatible fragments while preserving the policy semantics. The experimental results demonstrated the effectiveness of FNF in systematically removing rule order dependency and firewall rule anomalies while maintaining semantic equivalence and improving the structural quality of the rule set.

C. Incremental Rule Insertion

To simulate real-world firewall policy updates, we modeled the evolution process through incremental rule insertion, where rules were inserted in reverse order, starting from the last rule and proceeding sequentially toward the first. At each insertion step, we constructed two corresponding firewall rule sets with and without FNF. For the non-FNF rule set, the newly inserted rule was directly added to the rule set obtained from the previous insertion step. For the FNF-based rule set, the rule set obtained from the previous step was first transformed to satisfy all required normal forms, ensuring that all existing rules were mutually exclusive and free of overlap. After this normalization process, a new rule that had not been normalized was inserted into the rule set.

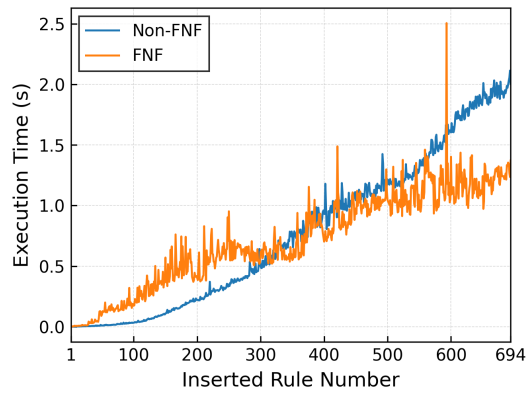


Fig. 4. Anomaly Detection Time

The key difference between the two settings lies in the structural properties of the rule sets, and this difference allowed us to use different anomaly detection procedures when measuring the anomaly detection time. For the non-FNF case, since overlaps could exist among all rules, anomaly detection had to consider all interactions between every pair of rules. In contrast, a rule set that satisfies 4FNF guarantees that all existing rules are mutually exclusive. Therefore, there are no anomalies between the existing rules. When a non-normalized rule is inserted into such a rule set, any resulting anomalies can only arise from interactions between the newly inserted rule and the existing rules. Therefore, anomaly detection only needs to examine the interactions between the newly inserted rule and each existing rule, without re-evaluating the relationships among previously processed rules.

The anomaly detection time at each rule insertion step for rule sets with and without FNF is shown in Fig. 4. In the early stages of rule insertion, the FNF-based rule set had a longer detection time compared to the non-FNF rule set. This was mainly due to overhead introduced by atomization in 1FNF, which outweighed the benefit of reduced detection comparisons when the rule set was still small. However, as the number of inserted rules increased, this trend gradually changed. After approximately 300 inserted rules, the detection time for the non-FNF rule set exceeded that of the FNF-based rule set. This crossover highlights the scalability advantage of FNF. While normalization introduces an initial overhead, it effectively controls the growth of detection cost as the rule set expands. For modern firewall rule sets with thousands of rules, the benefits of FNF become increasingly significant, making it a more efficient approach for long-term management.

V. CONCLUSION

This paper proposes a series of normal forms and a novel way to sequentially remove order dependency in a firewall. The effectiveness of the proposed normalization methods was evaluated through both static and dynamic experiments. The static experiment applied FNF to the data center dataset from Academia Sinica and showed that all anomalies caused by overlapping packet matching spaces in rules were successfully

corrected. The dynamic experiment sequentially inserted rules into an existing firewall to simulate policy evolution, and the results showed that a firewall maintained under FNF detected anomalies more efficiently than a firewall without FNF when a new rule was inserted. These results demonstrate that FNF provides an effective and principled approach for eliminating rule-order dependency, correcting firewall anomalies, and improving the efficiency of anomaly detection during the maintenance of the firewall policy.

REFERENCES

- [1] E. Al-Shaer and H. Hamed, "Firewall policy advisor for anomaly discovery and rule editing," in *IFIP/IEEE Eighth International Symposium on Integrated Network Management*, 2003., 2003, pp. 17–30.
- [2] L. Yuan, H. Chen, J. Mai, C.-N. Chuah, Z. Su, and P. Mohapatra, "Fireman: a toolkit for firewall modeling and analysis," in *2006 IEEE Symposium on Security and Privacy (SP'06)*, 2006, pp. 15 pp.–213.
- [3] M. Abedin, S. Nessa, L. Khan, and B. Thuraisingham, "Detection and resolution of anomalies in firewall policy rules," in *IFIP Annual Conference on Data and Applications Security and Privacy*. Springer, 2006, pp. 15–29.
- [4] H. Hu, G.-J. Ahn, and K. Kulkarni, "Detecting and resolving firewall policy anomalies," *IEEE Transactions on Dependable and Secure Computing*, vol. 9, no. 3, pp. 318–331, 2012.
- [5] M. Alicea and I. Alsmadi, "Misconfiguration in firewalls and network access controls: Literature review," *Future Internet*, vol. 13, no. 11, 2021. [Online]. Available: <https://www.mdpi.com/1999-5903/13/11/283>
- [6] P. Adão, C. Bozzato, G. D. Rossi, R. Focardi, and F. Luccio, "Mignis: A semantic based tool for firewall configuration," in *2014 IEEE 27th Computer Security Foundations Symposium*, 2014, pp. 351–365.
- [7] T. Wong, "On the usability of firewall configuration," in *Symposium on usable privacy and security*, 2008.
- [8] S. B. N. Ramez Elmasri, "Fundamentals of database systems 7th ed." 2016.
- [9] E. F. Codd, "A relational model of data for large shared data banks," *Commun. ACM*, vol. 13, no. 6, p. 377–387, Jun. 1970. [Online]. Available: <https://doi.org/10.1145/362384.362685>
- [10] J. Wack, K. Cutler, and J. Pole, "Guidelines on firewalls and firewall policy," 2002.
- [11] B. Fraser, "Site security handbook," Tech. Rep., 1997.
- [12] M. Jethanandani, S. Agarwal, L. Huang, and D. Blair, "YANG Data Model for Network Access Control Lists (ACLs)," RFC 8519, Mar. 2019. [Online]. Available: <https://www.rfc-editor.org/info/rfc8519>
- [13] M. Abedin, S. Nessa, L. Khan, E. Al-Shaer, and M. Awad, "Analysis of firewall policy rules using traffic mining techniques," *Int. J. Internet Protoc. Technol.*, vol. 5, no. 1/2, p. 3–22, Apr. 2010. [Online]. Available: <https://doi.org/10.1504/IJIP.2010.032611>
- [14] E. Al-Shaer and H. Hamed, "Discovery of policy anomalies in distributed firewalls," in *IEEE INFOCOM 2004*, vol. 4, 2004, pp. 2605–2616 vol.4.
- [15] H. Dhrir, M. Charfeddine, N. Tarhouni, and H. M. Kammoun, "Machine learning-and deep learning-based anomaly detection in firewalls: a survey," *The Journal of Supercomputing*, vol. 81, no. 6, p. 761, 2025.
- [16] C.-S. Lee and L.-J. Chen, "Enhancing firewall rule anomaly detection via llm alignment," in *International Conference on Technologies and Applications of Artificial Intelligence*. Springer, 2025.
- [17] E. F. Codd, "Further normalization of the data base relational model," *Data base systems*, vol. 6, no. 1972, pp. 33–64, 1972.
- [18] E. F. Codd, "Recent investigations in relational data base systems," in *ACM Pacific*, 1974. [Online]. Available: <https://api.semanticscholar.org/CorpusID:47325247>
- [19] R. Fagin, "Multivalued dependencies and a new normal form for relational databases," *ACM Trans. Database Syst.*, vol. 2, no. 3, p. 262–278, Sep. 1977. [Online]. Available: <https://doi.org/10.1145/320557.320571>
- [20] R. Fagin, "Normal forms and relational database operators," in *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '79. New York, NY, USA: Association for Computing Machinery, 1979, p. 153–160. [Online]. Available: <https://doi.org/10.1145/582095.582120>